Exercise Sheet 2

Graph Theory

5942UE Network Science

18 March 2026

Prof. Dr. Michael Granitzer Dr. Kanishka Ghosh Dastidar Dr. Jelena Mitrovic

Python Setup

Python Setup 1/2: Install Conda

We will use Python with NetworkX.

Install **Miniconda** from:

https://docs.conda.io/en/latest/miniconda.html

Choose your system:

- **Windows:** download the installer, run it, then open **Anaconda Prompt**
- **Mac:** download the `.pkg` installer and install normally
- **Linux:** download the `.sh` installer and run:

```
bash Miniconda3-latest-Linux-x86_64.sh
```

After installation, restart your terminal.

Python Setup 2/2: Create Environment

Check Conda:

```
conda --version
```

Create and activate the course environment:

```
conda create -n network-science python=3.11
conda activate network-science
conda install -c conda-forge networkx matplotlib numpy scipy pandas jupyter
```

Start Jupyter:

```
jupyter notebook
```

Test in a notebook:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()
nx.draw(G, with_labels=True)
plt.show()
```

If a graph appears, the setup works.



2.A Properties of Networks

Exercise 2.A.1: Two Meanings of the Same Network

The ARPANET had two distinct kinds of nodes: **universities/labs** and **routers**. Depending on which you model as nodes, you get different graphs of the same physical system.

- **Model R** (router-level): nodes are routers; edges are physical cables.
- **Model U** (institution-level): nodes are institutions; an edge connects two institutions if at least one router-level path exists between them.

1. For a path $A \to B$ in Model R, what does it mean in the physical system?
2. For the same path in Model U, what does it represent? How do the two models differ in what they express?
3. If your question is *"Can MIT reach Stanford in at most 3 hops?"*, which model do you use and why?
4. If your question is *"Which physical cable, if cut, would disconnect the ARPANET?"*, which model do you use and why?
5. What general principle do these questions illustrate about modeling choices?



Solution:

1. Model R path: A sequence of physical cable segments connecting two specific routers. Path length = number of cable hops.

2. Model U path: A sequence of institutions reachable from each other. This hides router-level detail — one Model U "hop" may correspond to dozens of physical cable segments.

3. MIT → Stanford, 3 hops: Use **Model U**. The question is about institutional reachability, not physical routing.

4. Critical cable: Use **Model R**. You need physical resolution. In Model U the cable is invisible.

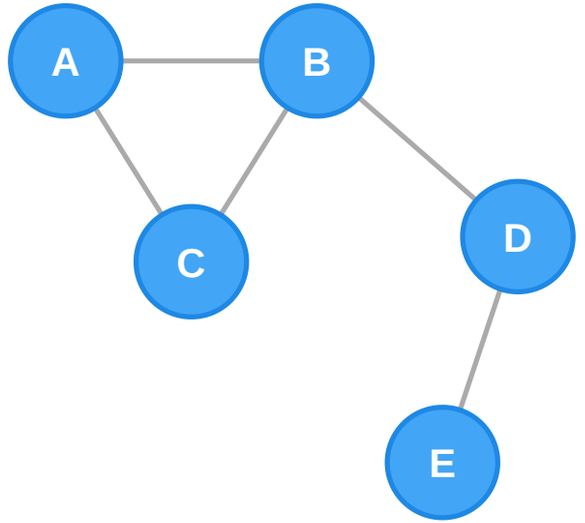
5. General principle: The right model depends on the question. Using a model at the wrong level of abstraction either gives the wrong answer or hides the relevant information.

2.B Representing Graphs

Exercise 2.B.1: From Picture to Formal Notation

Consider an undirected graph with five nodes $\{A, B, C, D, E\}$ and edges A–B, A–C, B–C, B–D, D–E.

1. Write the formal set notation: $V = \ldots, E = \ldots$
2. Draw the adjacency matrix (nodes in alphabetical order).
3. Write the adjacency list as a Python dictionary.
4. Compute $|V|$ and $|E|$.
5. What is the degree of each node? Verify the handshaking lemma: $\sum_{v \in V} \deg(v) = 2|E|$.
6. Now suppose B–D becomes directed ($B \to D$). What changes?



Solution:

1. $V = \{A, B, C, D, E\}, E = \{\{A, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \{D, E\}\}$

2. **Adjacency matrix** (order A, B, C, D, E):

$$M = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

3. **Adjacency list:**

```
{"A": ["B","C"], "B": ["A","C","D"], "C": ["A","B"],
 "D": ["B","E"],  "E": ["D"]}
```

4. $|V| = 5, |E| = 5.$

5. Degrees: $\deg(A) = 2,\ \deg(B) = 3,\ \deg(C) = 2,\ \deg(D) = 2,\ \deg(E) = 1.$
$\sum \deg = 10 = 2 \times 5 \checkmark$

6. Directed $B \to D$**:** Matrix loses symmetry. B's out-deg = 3, in-deg = 2; D's out-deg = 1, in-deg = 1.
Lemma: $\sum \text{in-deg} = \sum \text{out-deg} = |E|.$

Exercise 2.B.2: Graph Representations in Python

Task (10 min): Work with the graph from Exercise B.1 using NetworkX, the standard Python package for the creation, manipulation, and study of complex networks.

```
import networkx as nx

G = nx.Graph()
G.add_edges_from([("A","B"),("A","C"),("B","C"),("B","D"),("D","E")])
```

1. Print the adjacency matrix using `nx.to_numpy_array`.
2. Print the degree of each node. Does it match your hand calculation?
3. Compute and print the **density** of the graph. What does density = 1 mean? What does density = 0 mean?
4. Add a directed version: `DG = nx.DiGraph()`. Print in-degree and out-degree of B and D.
5. **Bonus:** Plot both graphs side by side with `matplotlib`.

Solution:

```python
import networkx as nx

G = nx.Graph()
G.add_edges_from([("A","B"),("A","C"),("B","C"),("B","D"),("D","E")])

# 1. Adjacency matrix
nodes = sorted(G.nodes())
print("Adjacency matrix:\n", nx.to_numpy_array(G, nodelist=nodes))

# 2. Degrees
print("Degrees:", dict(G.degree()))
# {'A': 2, 'B': 3, 'C': 2, 'D': 2, 'E': 1}

# 3. Density
print(f"Density: {nx.density(G):.3f}")   # 0.500
```

Density = 1 → complete graph; **density = 0** → no edges.

Solution (continued):

```
# 4. Directed version
DG = nx.DiGraph()
DG.add_edges_from([
    ("A", "B"), ("A", "C"), ("B", "C"),
    ("B", "D"), ("D", "E"),
])
print("B:", DG.in_degree("B"),  DG.out_degree("B"))
print("D:", DG.in_degree("D"),  DG.out_degree("D"))
```

The directed version loses matrix symmetry; B's high out-degree reflects its role as a "broadcaster".



Exercise 2.B.3: Reading a Network Diagram

Task (10 min): Use Python and NetworkX to build the Zachary karate club graph — one of the most studied social networks — and answer the following programmatically:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()
```

1. How many nodes and edges does the network have?
2. What is the average degree? What does this mean for the "average member"?
3. Are there any isolated nodes (degree 0)? What would an isolated node mean socially?
4. Plot the network. Can you visually identify the two factions that the club split into?

Hint: `G.degree()` returns degree for all nodes. `nx.draw_spring` gives a reasonable layout.

Solution:

```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()

print(f"Nodes: {G.number_of_nodes()}")       # 34
print(f"Edges: {G.number_of_edges()}")       # 78

degrees = [d for _, d in G.degree()]
avg_degree = sum(degrees) / len(degrees)
print(f"Average degree: {avg_degree:.2f}")   # 4.59

isolated = [n for n, d in G.degree() if d == 0]
print(f"Isolated nodes: {isolated}")          # []
```

- **34 nodes, 78 edges:** small but dense.
- **Average degree ≈ 4.6:** each member is friends with 4–5 others.
- **No isolated nodes:** every member has at least one friend — connected.

Solution (continued):

```
color_map = ["steelblue" if G.nodes[n]["club"] == "Mr. Hi"
             else "tomato" for n in G.nodes()]
nx.draw_spring(G, node_color=color_map, with_labels=True,
               node_size=400, font_size=8)
plt.title("Zachary Karate Club – two factions")
plt.show()
```

- **Visual factions:** the spring layout reveals two clusters; high-degree hubs ("Mr. Hi" and "Officer") are structurally central.



2.C Paths and Cycles

Exercise 2.C.1: Walks, Paths, and Cycles

Consider the undirected graph with edges $\{A–B,\ B–C,\ C–D,\ D–A,\ A–C\}$.

Classify each sequence as a **walk**, **path**, **cycle**, or **none** (edge doesn't exist). Justify each answer.

Sequence	Classification	Reason
A, B, C, A	?	?
A, C, D, A	?	?
A, B, C, D, A	?	?
A, B, A, C	?	?
A, B, C, D, C	?	?
A, E	?	?

Then: what is the **diameter** of this graph?



Solution:

A **cycle** is a path that starts and ends at the same vertex, with no other repeated vertices.

Sequence	Classification	Reason
A, B, C, A	**Cycle**	Closed path; only A repeats as start/end; all edges exist
A, C, D, A	**Cycle**	Closed path; only A repeats as start/end; all edges exist
A, B, C, D, A	**Cycle**	Closed path; only A repeats as start/end; all edges exist
A, B, A, C	**Walk** (not path/cycle)	All edges exist, but A repeats before the end
A, B, C, D, C	**Walk** (not path/cycle)	All edges exist, but C repeats and the walk is not closed
A, E	**None**	Node E does not exist

Diameter: $d(A, B) = 1, d(A, C) = 1, d(A, D) = 1, d(B, C) = 1, d(B, D) = 2, d(C, D) = 1$. **Diameter = 2.**

Exercise 2.C.2: The Königsberg Bridge Problem

The Königsberg bridges problem (Euler, 1736) asks: can you walk through the city crossing each of the seven bridges exactly once and return to your starting point?

The city has four land masses: **North bank (N)**, **South bank (S)**, **Island (I)**, **East bank (E)**. Bridges: N–I (×2), S–I (×2), N–E (×1), S–E (×1), I–E (×1).

1. Model this as a multigraph $G = (V, E)$. Write down V and E.
2. Write the degree of each node.
3. Euler proved that an **Eulerian circuit** exists iff **every vertex has even degree**. Does Königsberg satisfy this condition?
4. An **Eulerian trail** (open or closed) exists iff **exactly 0 or 2 vertices have odd degree**. With 0 odd-degree vertices, the trail is an Eulerian circuit; with 2, it is open. Does Königsberg have an Eulerian trail?
5. What does this tell us?





Solution:

1. Vertices: $V = \{N, S, I, E\}$.

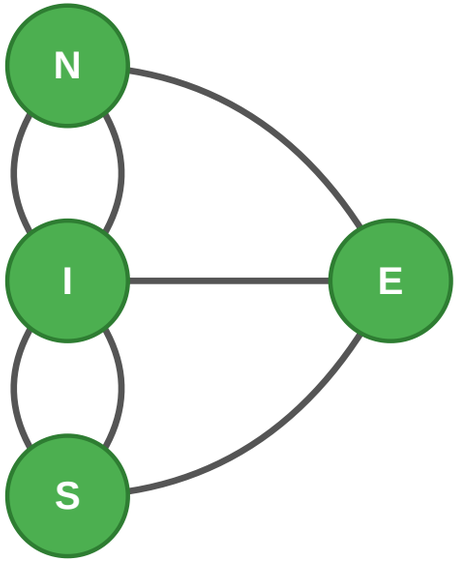
Edges with multiplicity:

- between N and I: 2 bridges
- between S and I: 2 bridges
- between N and E, S and E, I and E: 1 bridge each

Thus $|E| = 7$.

2. Degrees:

- $\deg(N) = 3, \deg(S) = 3$
- $\deg(I) = 5, \deg(E) = 3$



3. Eulerian circuit: All degrees must be even. Here all four are odd → **no Eulerian circuit**.

4. Eulerian trail: Requires exactly 0 or 2 odd-degree vertices. All four are odd → **no Eulerian trail** either.

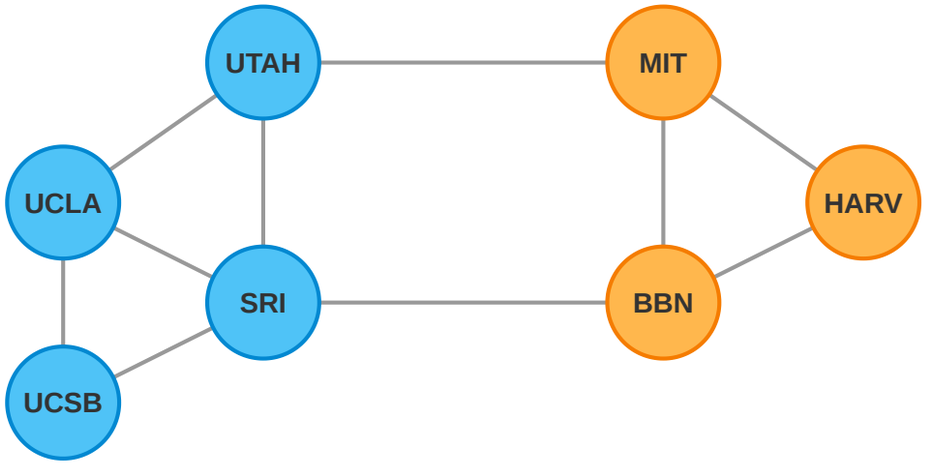
5. Lesson: Euler's result reduces a physical puzzle to a parity check. The actual shape of the river and bridge positions are irrelevant. This is the power of graph abstraction.

Exercise 2.C.3: Shortest Paths with NetworkX

Task (10 min): The ARPANET had 20 nodes in its early topology. We will work with a small synthetic version.

```
import networkx as nx
import matplotlib.pyplot as plt

arpa = nx.Graph()
arpa.add_edges_from([
    ("UCLA", "SRI"), ("UCLA", "UCSB"), ("UCLA", "UTAH"),
    ("SRI",  "UTAH"), ("SRI",  "BBN"),
    ("UCSB", "SRI"), ("UTAH", "MIT"),
    ("BBN",  "HARVARD"), ("BBN",  "MIT"), ("MIT", "HARVARD"),
])
```



1. Plot the network using `nx.draw_spring()`.
2. Find the shortest path from `"UCLA"` to `"HARVARD"`. Print path and length.
3. Compute eccentricities, **diameter**, and **radius**.
4. Which node is the center? What does this mean structurally?
5. Remove `"BBN"` and recompute the diameter. What changed and why?





Solution:

```
# 1. Plot
nx.draw_spring(arpa, with_labels=True, node_color='lightblue')
plt.show()

# 2. Shortest path
path = nx.shortest_path(arpa, "UCLA", "HARVARD")
print(f"Path: {path}")           # ['UCLA', 'SRI', 'BBN', 'HARVARD']

# 3. Eccentricity, diameter, radius
print(f"Diameter: {nx.diameter(arpa)}")  # 4
print(f"Radius:   {nx.radius(arpa)}")    # 2

# 4. Center
center = nx.center(arpa)
print(f"Center: {center}")

# 5. Remove BBN
arpa2 = arpa.copy()
```

Interpretation: The center node(s) minimise the maximum detour any message must take. Removing BBN increases the diameter because it was a primary relay bridging the East Coast and West Coast clusters.

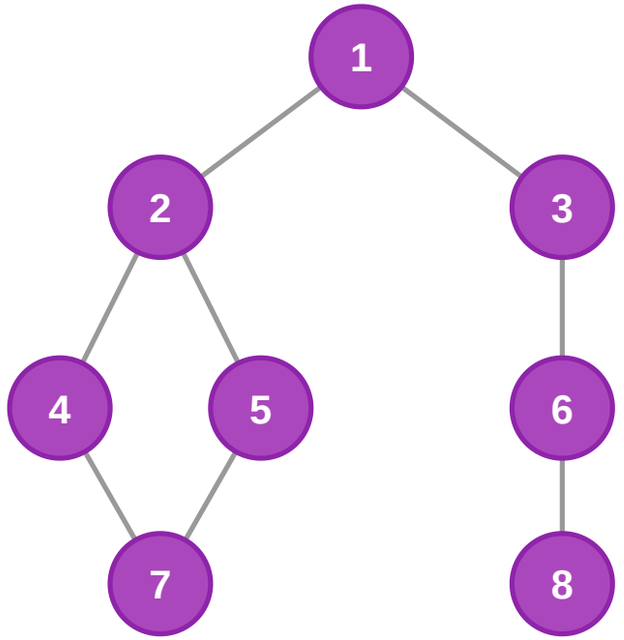
2.D Breadth-First Search (BFS)

Exercise 2.D.1: BFS by Hand — Layer Tracing

Run BFS starting from node 1 on the following undirected graph:

Edges: $\{1\text{–}2, 1\text{–}3, 2\text{–}4, 2\text{–}5, 3\text{–}6, 4\text{–}7, 5\text{–}7, 6\text{–}8\}$

1. Fill in the BFS discovery table with layer and parent for each node.
2. Draw the BFS tree.
3. What is $d(1, 7)$ and $d(1, 8)$? Is there more than one shortest path to 7?
4. If we add weights $w(2\text{–}4) = 10, w(5\text{–}7) = 1$, does BFS still give shortest paths?



Solution:

Node	Layer	Parent
1	0	—
2, 3	1	1
4, 5	2	2
6	2	3
7, 8	3	4 or 5; 6

Node 7 can have either 4 or 5 as a parent depending on tie-breakers, showing more than one shortest path.

- **Shortest paths:** $d(1, 7) = 3, d(1, 8) = 3$.
- **Weighted BFS:** No. BFS counts hops, not weights. Use Dijkstra for weighted shortest paths.

Exercise 2.D.2: Implementing BFS and Computing Distances

Task (15 min): Implement BFS from scratch and use it to answer structural questions about the Zachary karate club network.

```
from collections import deque
import networkx as nx

def bfs_distances(graph, source):
    """Return a dict {node: shortest-path distance from source}."""
    dist = {source: 0}
    queue = deque([source])
    while queue:
        node = queue.popleft()
        # --- your code here ---
    return dist

G = nx.karate_club_graph()
adj = {n: list(G.neighbors(n)) for n in G.nodes()}
```

1. Complete `bfs_distances`. Verify it against `nx.single_source_shortest_path_length`.
2. Use it to find the **eccentricity** of node 0 (the instructor "Mr. Hi").
3. What fraction of the network is reachable within 2 hops from node 0?
4. Repeat for node 33 (the administrator "Officer"). Compare eccentricities and reachable fractions. What does the difference tell you?

Solution:

```
from collections import deque
import networkx as nx

def bfs_distances(graph, source):
    dist = {source: 0}
    queue = deque([source])
    while queue:
        node = queue.popleft()
        for neighbor in graph.get(node, []):
            if neighbor not in dist:
                dist[neighbor] = dist[node] + 1
                queue.append(neighbor)
    return dist

G = nx.karate_club_graph()
adj = {n: list(G.neighbors(n)) for n in G.nodes()}

d0  = bfs_distances(adj, 0)
```

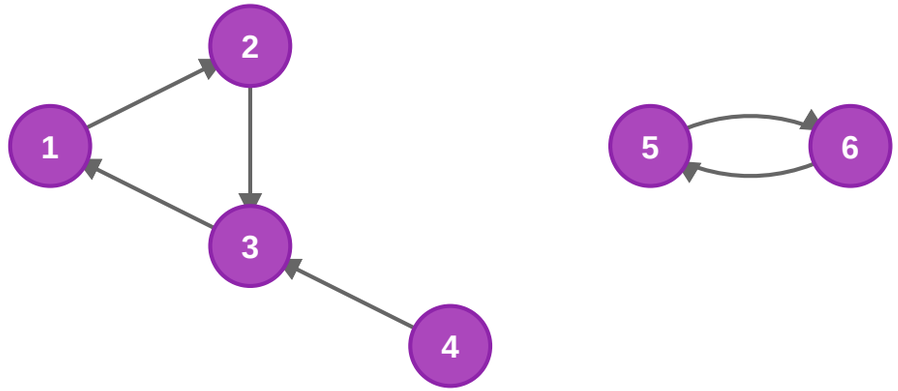
- **Eccentricity of Mr. Hi (0):** ≈ 4. Reaches more nodes in 2 hops than node 33.
- **Structural insight:** Node 0's central position is reflected in lower BFS distances. The two factions are anchored by these hubs (0 and 33), but Mr. Hi is often more "central" in the unweighted friendship graph.

2.E Connectivity and Components

Exercise 2.E.1: Components and Connectivity by Hand

Consider a directed graph with nodes $\{1, 2, 3, 4, 5, 6\}$ and directed edges: $1 \to 2,\ 2 \to 3,\ 3 \to 1,\ 4 \to 3,\ 5 \to 6,\ 6 \to 5$.

1. List all **weakly connected components** (ignoring direction).
2. List all **strongly connected components** (following directed paths).
3. Node 4 is a "source" node. Explain what this means for information flow. Can node 4 receive information from nodes 1, 2, or 3?
4. If you removed edge $3 \to 1$, how would the SCCs change?



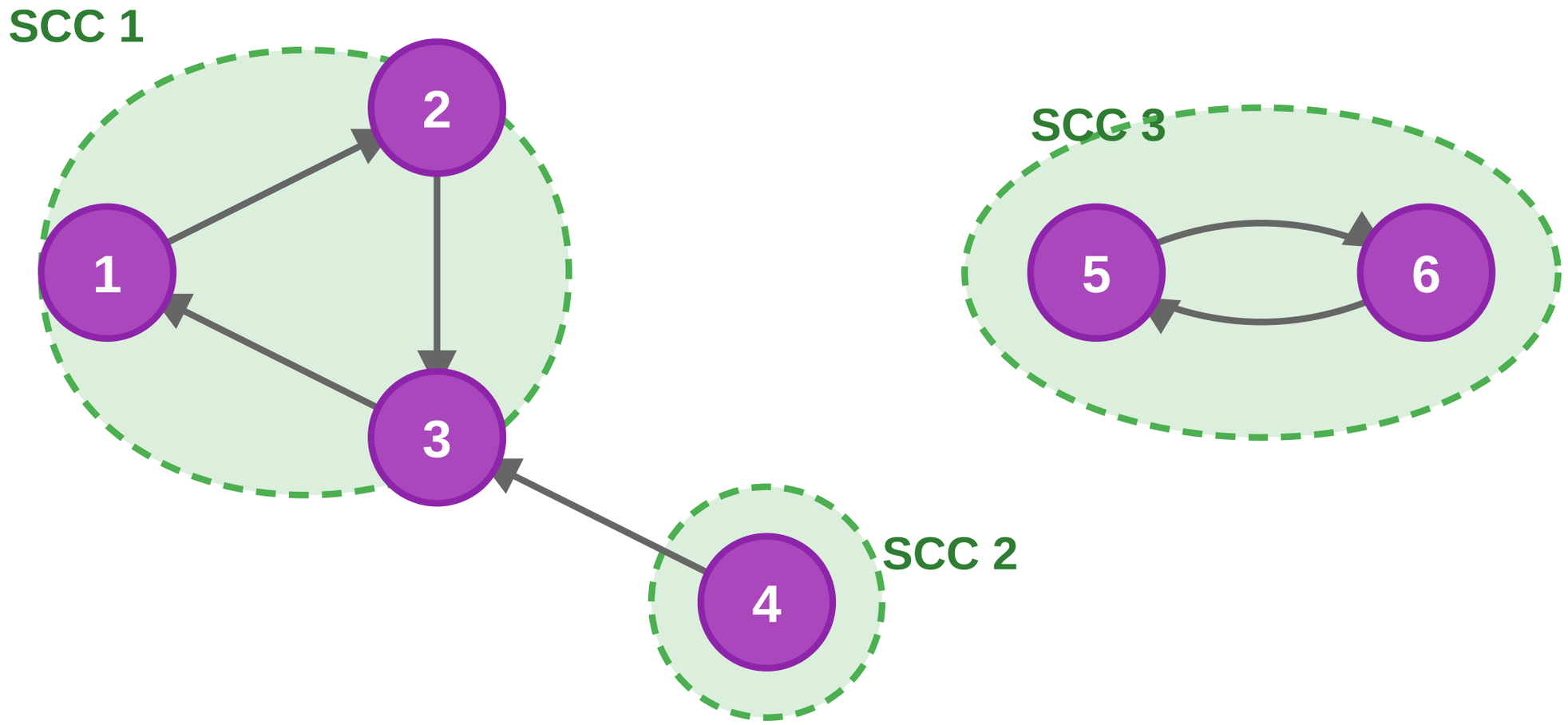
Solution:

1. Weakly connected components (undirected view):

- $\{1, 2, 3, 4\}$
- $\{5, 6\}$

2. Strongly connected components (directed paths):

- $\{1, 2, 3\}$ — cycle $1 \to 2 \to 3 \to 1$
- $\{4\}$ — reaches $\{1, 2, 3\}$ via $4 \to 3$ but no path back
- $\{5, 6\}$ — cycle $5 \to 6 \to 5$



3. Node 4 as source: Information flows out of 4 into $\{1, 2, 3\}$ but nothing flows back. Node 4 cannot receive information from 1, 2, or 3. It acts as a "broadcaster" influencing the main cluster without

Exercise 2.E.2: Giant Components in Practice

Task (10 min): Use NetworkX to investigate what happens to connectivity when we progressively remove nodes from the karate club network.

```
import networkx as nx
import matplotlib.pyplot as plt
import random

G = nx.karate_club_graph()
```

1. What fraction of nodes belong to the giant component of the original network?
2. **Random failure:** repeatedly remove a randomly chosen node and record the size of the largest component after each removal. Plot the result.
3. **Targeted attack:** repeatedly remove the node with the **highest current degree**. Plot on the same axes.
4. Which strategy destroys the giant component faster? Why?

Hint: `max(nx.connected_components(G), key=len)` gives the largest component.

Solution:

```
import networkx as nx
import matplotlib.pyplot as plt
import random

def giant_fraction(G):
    if G.number_of_nodes() == 0:
        return 0
    return len(max(nx.connected_components(G), key=len)) / G.number_of_nodes()

def simulate(G0, strategy):
    G = G0.copy()
    sizes = [giant_fraction(G)]
    while G.number_of_nodes() > 0:
        if strategy == "random":
            v = random.choice(list(G.nodes()))
        else:  # targeted
            v = max(G.nodes(), key=lambda n: G.degree(n))
        G.remove_node(v)
```

- **Robustness vs fragility:** Hub-based networks are resilient to random failure (most nodes are peripheral) but fragile under targeted attack (hubs are critical).
- Removing the instructor (0) or administrator (33) disrupts connectivity far more than removing an average student. This principle explains why internet backbones and social hubs are high-value targets.